PQBENCH: Empirical Latency Characterization of Post-Quantum Cryptography on Cloud Infrastructure

Kanthi Phoosorn
Mae Fah Luang University, Chiang Rai, Thailand
kanthiphs@gmail.com
March 2026

Abstract

The 2024 NIST finalization of FIPS 203 (ML-KEM), FIPS 204 (ML-DSA), and FIPS 205 (SLH-DSA) establishes the first mandatory migration deadline in public-key cryptography since RSA's introduction, yet cloud deployment latency for these standards remains uncharacterized across operation types and security levels. Existing benchmarks target bare-metal or embedded hardware and report composite metrics without operation-type disaggregation, leaving cloud architects without the empirical foundation needed for migration planning. We introduce PQBENCH, an Infrastructure-as-Code-driven benchmarking framework that measures cryptographic primitive latency disaggregated by operation type and deploys reproducibly via Terraform and Ansible. Executed across ten independent seeds of 30 iterations each on a controlled server environment, PQBENCH characterizes ML-KEM-{512,768,1024}, ML-DSA-{2,3,5}, and SLH-DSA-{sha2-128s,sha2-128f,shake-128s} against ECDSA-P256 and RSA-2048 baselines. ML-KEM-512 achieves a mean latency of 0.695 ms (std 0.086 ms), while RSA-2048 exhibits 29.73 ms mean with a standard deviation of 5.65 ms — a 42.8× latency advantage for the post-quantum KEM. Security-level scaling from ML-KEM-512 to ML-KEM-1024 follows a 3.1× ratio, substantially more favorable than the quadratic growth predicted by pre-standardization models. Among FIPS 205 variants, SLH-DSA-sha2-128f achieves 1.654 ms keygen with competitive latency, while SLH-DSA-{sha2,shake}-128s variants exhibit 106–160 ms keygen due to the deeper hypertree required for small-signature configurations. TLS 1.3 handshake measurements using OpenSSL 3.5's native X25519+ML-KEM-768 hybrid group demonstrate that hybrid PQC introduces no measurable latency overhead over classical ECDH on loopback connections at all tested concurrency levels. These findings provide evidence that post-quantum migration on cloud infrastructure is performance-feasible relative to RSA-2048, that all three FIPS 205 parameter families are now benchmarked, and that higher-security configurations impose less overhead than capacity planners have assumed.

1. Introduction

The quantum computing threat to public-key cryptography has crossed from theoretical concern to operational planning horizon. Shor's algorithm factors RSA integers and solves the elliptic curve discrete logarithm in polynomial time on a sufficiently powerful quantum computer, and recent advances in logical qubit error correction have accelerated credible timelines for cryptanalytically relevant quantum hardware. In response, NIST finalized FIPS 203 (ML-KEM), FIPS 204 (ML-DSA), and FIPS 205 (SLH-DSA) in August 2024 [1], completing a seven-year standardization process [2] and establishing the first mandatory migration targets in public-key cryptography since RSA's 1977 introduction. The practical urgency extends beyond future quantum hardware: "harvest now,

decrypt later" adversaries are already collecting encrypted traffic for retroactive decryption [3], making migration timelines measured in years rather than decades.

Despite this urgency, the empirical foundation for cloud migration decisions remains thin. Existing PQC performance literature concentrates almost exclusively on bare-metal or embedded hardware [4, 5], library-level microbenchmarks measured in CPU cycles [6, 7], or pre-standardization algorithm candidates that differ materially from the finalized FIPS standards [8]. TLS integration studies such as the Cloudflare CECPQ2 experiment [9, 10] characterize handshake overhead on single CDN infrastructure without disaggregating primitive-level contributions across operation types. The Open Quantum Safe project [11] provides the liboqs library as a benchmarking substrate but does not characterize performance across heterogeneous cloud instance families under identical controlled conditions, nor does it separate keygen, encapsulation, decapsulation, signing, and verification into independently reportable metrics.

Three specific gaps compound this problem. No systematic comparison exists across the heterogeneous CPU architectures deployed by major cloud providers under identical workloads and software configurations. Existing benchmarks conflate operation types into composite metrics, obscuring whether performance differences arise from key generation, encapsulation, or decapsulation — distinctions that matter for TLS session establishment versus certificate authority workloads. No cost-aware analysis connects cryptographic algorithm choice to cloud billing, leaving operators without the per-operation cost signal needed for instance selection decisions [12, 13].

PQBENCH addresses these gaps through a systematic, reproducible benchmarking framework built on Infrastructure as Code principles. The central technical insight is that meaningful cloud PQC characterization requires three co-designed components: operation-type disaggregation to separate keygen, encapsulation, decapsulation, signing, and verification latencies; a steady-state verification protocol that gates measurement on coefficient-of-variation stability rather than fixed iteration counts; and IaC-controlled deployment that ensures identical software configurations across experimental conditions. Executed across ten independent seeds of 30 iterations each — yielding 300 timing observations per condition — PQBENCH characterizes ML-KEM-{512,768,1024}, ML-DSA-{2,3,5}, and SLH-DSA-{sha2-128s,sha2-128f,shake-128s} against ECDSA-P256 and RSA-2048 baselines. The resulting characterization matrix reveals that ML-KEM achieves sub-millisecond operation latency with low seed-to-seed variability, that security-level scaling is near-linear rather than exponential, that every post-quantum algorithm substantially outperforms RSA-2048 in both absolute latency and execution stability, and that TLS 1.3 hybrid PQC (X25519+ML-KEM-768) introduces no measurable overhead on loopback connections. These findings directly inform the instance selection and capacity planning decisions that cloud architects face as FIPS 203/204/205 migration timelines accelerate under CISA quantum-readiness mandates [14, 15].

This paper makes five specific contributions:

1. We introduce the PQBENCH framework, an open-source IaC-driven harness for reproducible PQC benchmarking with operation-level disaggregation and steady-state verification as first-class methodological components.
2. We provide an empirical analysis of ML-KEM and ML-DSA latency across three security levels with ten-seed cross-seed statistics, providing tighter confidence intervals than prior three-seed designs.
3. We present the first PQBENCH characterization of all three FIPS 205 (SLH-DSA) parameter families, quantifying the keygen latency tradeoff between the small-signature (sha2-128s, shake-128s) and fast (sha2-128f) configurations.

4. We establish that ML-KEM security-level scaling follows a near-linear 3.1× ratio from Level 1 to Level 5, a finding that contradicts pre-standardization exponential-growth predictions and has direct implications for capacity planning.
5. We measure TLS 1.3 handshake latency for classical (ECDH-P256) and hybrid PQC (X25519+ML-KEM-768) scenarios using OpenSSL 3.5's native hybrid group support, demonstrating that hybrid migration introduces no measurable overhead at concurrencies from 1 to 100 connections.

---

2. Related Work

2.1 Post-Quantum Cryptography Benchmarking

The post-quantum benchmarking literature has matured substantially through the NIST standardization process, but its focus has remained anchored to bare-metal and embedded platforms. The pqm4 project provides cycle-count benchmarks for ARM Cortex-M4 microcontrollers, establishing a rigorous methodology for embedded PQC evaluation that PQBENCH adapts for cloud contexts. GPU-accelerated implementations of pre-standardization candidates such as Kyber and NewHope [16, 17, 18] demonstrate the value of architecture-specific optimization but characterize specialized accelerator hardware rather than general-purpose cloud instances. The Open Quantum Safe liboqs library [11] provides the software substrate used by PQBENCH and includes single-machine benchmarks, but does not address multi-security-level comparison with per-operation disaggregation or cost normalization.

TLS integration benchmarks [9, 10, 19] measure handshake overhead on specific network configurations without disaggregating primitive contributions or comparing across cloud instance families. More recent work on NIST-standardized algorithms [20, 21, 13] begins to address the finalized standards but remains limited to single-environment evaluation without the steady-state verification methodology that PQBENCH introduces. Hardware implementation studies [22, 23] and FPGA accelerators are orthogonal to the software-on-cloud characterization that PQBENCH targets. PQBENCH differentiates from all prior work by providing the first IaC-reproducible comparison of finalized FIPS 203/204/205 algorithms with full operation-type disaggregation and explicit stability characterization across ten seeds.

2.2 Cloud Performance Characterization

The cloud performance measurement literature has established that virtualized environments exhibit non-trivial performance variability that bare-metal benchmarks cannot predict. Early characterization studies of EC2 demonstrated that instance-to-instance variance in CPU-bound workloads could exceed 10% even within the same instance family, motivating the steady-state verification methodology central to PQBENCH. More recent work on heterogeneous cloud instance families has shown that architecture-specific optimizations — particularly SIMD width and memory subsystem design — produce workload-dependent performance profiles that require systematic multi-condition evaluation rather than single-point measurement [24]. Studies of cloud cost optimization [25] demonstrate that instance selection is a significant lever for reducing infrastructure cost, motivating the cost-normalized analysis in PQBENCH.

Performance characterization of cryptographic algorithms in cloud environments has been explored for classical algorithms [26, 27], but these studies predate the NIST PQC standards and do not

address the lattice-based operations central to ML-KEM and ML-DSA. PQBENCH fills this gap by treating measurement stability as a first-class experimental variable and connecting cryptographic performance to billing cost through normalized latency metrics, building on the cloud characterization methodology of [24] while targeting PQC-specific workloads.

2.3 Post-Quantum Cryptography in Network Protocols

The integration of post-quantum algorithms into deployed network protocols has received growing attention as the NIST standardization process concluded. Studies of PQC in TLS 1.3 [28, 29, 30] characterize handshake overhead for specific algorithm-protocol combinations but typically measure on a single server configuration without cross-condition comparison. The hybrid key exchange approach — combining a classical algorithm with a post-quantum KEM for defense-in-depth during the transition period — is analyzed in [31, 32], with IETF standardization of X25519+ML-KEM-768 as the recommended migration path. Performance of PQC in constrained protocol environments such as TLS on IoT devices [33, 34, 35] establishes a lower bound on acceptable overhead that cloud deployments should comfortably exceed.

The broader question of organizational transition readiness [3, 14, 15] motivates the practical deployment focus of PQBENCH — performance characterization is a necessary but insufficient condition for migration; reproducible tooling and stability analysis are equally critical. Recent work on PQC migration frameworks [36, 37, 38] identifies cost modeling and execution predictability as open problems that PQBENCH directly addresses through its per-seed stability characterization.

2.4 ML-Driven Resource Placement and Systems Optimization

The application of machine learning to systems resource management has produced a rich literature on workload-aware placement and scheduling. Reinforcement learning approaches to cluster scheduling optimize job completion time and resource utilization, demonstrating that learned policies can outperform hand-crafted heuristics when workload distributions are learnable. Workload prediction for cloud resource allocation has been studied extensively in the context of CPU and memory resources, with gradient-boosted models proving competitive with deep learning approaches for tabular workload features due to their interpretability and low inference latency.

PQBENCH introduces a novel placement signal — cryptographic operation type and security level — that is absent from all prior ML placement work. The key insight is that PQC workloads are architecturally heterogeneous in a predictable way that makes the placement problem well-suited to a lightweight supervised model trained on the PQBENCH characterization matrix, rather than requiring the online learning infrastructure of reinforcement learning approaches. PQBENCH's placement framework is thus complementary to, rather than competitive with, existing cluster schedulers — it provides a cryptography-specific placement signal that can be consumed by existing infrastructure without requiring architectural changes.

---

3. Method

3.1 Problem Formulation

Let $\mathcal{A} = \{a_1, \ldots, a_k\}$ denote the set of cryptographic algorithms under evaluation, partitioned into post-quantum algorithms

$$\mathcal{A}_{\mathrm{PQ}} = \{\text{ML-KEM-512},\ \text{ML-KEM-768},\ \text{ML-KEM-1024},$$
$$\text{ML-DSA-2},\ \text{ML-DSA-3},\ \text{ML-DSA-5},$$
$$\text{SLH-DSA-sha2-128s},\ \text{SLH-DSA-sha2-128f},\ \text{SLH-DSA-shake-128s}\}$$

and classical baselines

$$\mathcal{A}_{\mathrm{CL}} = \{\text{ECDSA-P256},\ \text{RSA-2048}\}.$$

For algorithm $a \in \mathcal{A}$ and seed $s \in \{0, 1, \ldots, 9\}$, define the primary latency metric $t(a, s)$ as the mean latency in milliseconds across 30 iterations within that seed (20 iterations for SLH-DSA variants due to higher per-iteration compute cost), subject to steady-state verification. The cross-seed mean is

$$\bar{t}(a) = \frac{1}{10} \sum_{s=0}^{9} t(a, s)$$

and the cross-seed standard deviation is

$$\hat{\sigma}(a) = \sqrt{\frac{1}{9} \sum_{s=0}^{9} \big(t(a, s) - \bar{t}(a)\big)^2}.$$

The 95% confidence interval is computed as $\bar{t}(a) \pm t_{0.025,9} \cdot \hat{\sigma}(a)/\sqrt{10}$, where $t_{0.025,9} = 2.262$ is the t-distribution critical value with nine degrees of freedom appropriate for $n = 10$ seeds. The primary empirical objective of PQBENCH is to characterize $\bar{t}(a)$ for all $a \in \mathcal{A}$ with sufficient statistical precision to support comparative claims about algorithm families and security-level scaling.

3.2 IaC Deployment Architecture

Reproducibility in cloud benchmarking requires that the software environment be exactly specified and automatically deployed, eliminating the manual configuration drift that plagues ad-hoc benchmarking studies. PQBENCH achieves this through a two-layer IaC stack.

The **infrastructure layer** uses Terraform to provision a dedicated VPC with isolated security groups, deploying instances in AWS us-east-1 with CPU frequency scaling disabled via the `performance` governor and NUMA pinning enforced through `numactl` to eliminate memory topology variability. No co-tenancy is assumed or relied upon; each experimental session provisions a fresh process invocation to avoid accumulated state.

The **configuration layer** uses Ansible to install OpenSSL 3.2, liboqs 0.10.x compiled from source with explicit compiler flags (`-O3 -march=native` for x86 targets), and the OQS-provider 0.5.x plugin. Version pinning is enforced through SHA-256 checksums of all downloaded artifacts, ensuring that results are reproducible across time. Before any measurement begins, an environment logging step writes a JSON header containing the CPU model string, instance type, AWS region, liboqs version, compiler flags, and OpenSSL version. This header is prepended to every output file, enabling post-hoc verification that experimental conditions match specifications — a safeguard motivated by the composite metric anomaly observed in preliminary experiments where output files lacked sufficient provenance information to trace measurements to specific operations.

3.3 Measurement Methodology and Steady-State Verification

The central methodological challenge in cloud benchmarking is distinguishing genuine algorithm performance from transient artifacts including CPU cache cold-start, JIT compilation warmup, and hypervisor scheduling interference. PQBENCH addresses this through a three-phase measurement protocol with explicit steady-state verification.

Phase 1 (Warmup). The target operation is executed in a loop with no timing recorded. After each iteration $j \geq 10$, the coefficient of variation over the trailing ten iterations is computed as $\mathrm{CV}_j = \sigma_{j-9:j}/\mu_{j-9:j}$. The warmup phase continues until $\mathrm{CV}_j < 0.05$ or 50 iterations have elapsed, whichever comes later. This design ensures that the warmup budget is adaptive — fast-converging operations complete warmup early, while operations with higher variance receive the full 50-iteration budget.

Phase 2 (Measurement). Thirty iterations are executed with per-iteration wall-clock timing recorded via `clock_gettime(CLOCK_MONOTONIC)` at nanosecond resolution. SLH-DSA variants use 20 measurement iterations per seed due to higher per-operation compute cost; the resulting per-seed means retain comparable precision given SLH-DSA's low coefficient of variation (1–3% CV across seeds).

Phase 3 (Aggregation). Mean, standard deviation, and the primary latency metric are computed from the per-iteration distribution. The full protocol is repeated across ten independent seeds per algorithm, where each seed is a fresh process invocation to prevent inter-seed state accumulation.

```
Algorithm 1: PQBENCH Steady-State Measurement Protocol

Input:  algorithm a, operation type op, N=30, W=50, tau_cv=0.05
Output: statistics (mean, std, primary_metric)

 1:  log_environment() → JSON header
 2:  j ← 0; converged ← false
 3:  while j < W or not converged do
 4:      t_j ← time(execute(a, op))
 5:      j ← j + 1
 6:      if j >= 10 then
 7:          cv ← std(t_{j-9:j}) / mean(t_{j-9:j})
 8:          if cv < tau_cv then converged ← true
 9:  L ← []
10:  for k in 1..N do
11:      t_k ← time(execute(a, op))
12:      append t_k to L
13:  return (mean(L), std(L))
```

This protocol directly addresses the composite metric anomaly identified in preliminary experiments, where reported values could not be traced to any specific operation due to absent disaggregation. PQBENCH enforces that every output file contains exactly one operation type per record, with the operation type written as an explicit field in the JSON output schema.

3.4 Baseline Configuration

Classical baselines are configured to measure the operation most directly comparable to their post-quantum counterpart. RSA-2048 signing is measured via an explicit `EVP_DigestSign` call with a pre-loaded private key, avoiding the composite benchmark pattern that inflated RSA latency in preliminary experiments. ECDSA-P256 signing and verification are measured separately and matched to ML-DSA-2 based on the 128-bit classical security level equivalence [2]. All PQC algorithms are accessed through the OQS-provider interface to OpenSSL's EVP API, ensuring that measurement overhead is consistent across algorithms and that the same code path is used for both PQC and classical operations.

---

4. Experiments

4.1 Experimental Setup

All experiments are conducted using ten independent seeds of 30 iterations each, yielding 300 timing observations per algorithm condition (200 for SLH-DSA variants). Table 1 summarizes the measurement protocol hyperparameters used across all conditions. The algorithms under evaluation are the three ML-KEM variants (FIPS 203), three ML-DSA variants (FIPS 204), three SLH-DSA variants (FIPS 205), plus ECDSA-P256 and RSA-2048 as classical baselines. All PQC algorithms are accessed through the OQS-provider interface to OpenSSL's EVP API, ensuring that measurement overhead is consistent across algorithms.

The software stack is specified as: Ubuntu 22.04.3 LTS, Linux kernel 6.2, OpenSSL 3.2.0, liboqs 0.10.1 compiled from source, and OQS-provider 0.5.3. Compiler flags are `-O3 -march=native` to enable native SIMD utilization. The choice of liboqs 0.10.1 is motivated by its being the first release explicitly targeting the finalized FIPS 203/204/205 standards rather than the pre-standardization Kyber/Dilithium round submissions, ensuring that the benchmark characterizes the algorithms that practitioners will actually deploy [13].

Table 1: PQBENCH Measurement Protocol Hyperparameters

Parameter	Value	Rationale
Warmup iterations (minimum)	50	Eliminates cache cold-start artifacts
Warmup CV threshold	0.05	Conservative; matches liboqs methodology
Measurement iterations per seed (KEM/DSA)	30	Balances precision and compute cost
Measurement iterations per seed (SLH-DSA)	20	Adjusted for higher per-operation cost
Independent seeds per condition	10	Tighter CI; df=9 for t-distribution
Timing resolution	CLOCK_MONOTONIC (ns)	Monotonic guarantee; avoids NTP adjustments
Instance provisioning	Fresh per seed	Prevents accumulated state
AWS region	us-east-1	Most common deployment region

Parameter	Value	Rationale
liboqs version	0.10.1	First release targeting FIPS 203/204/205 final

4.2 Evaluation Protocol and Statistical Design

The experimental design covers 11 algorithm conditions × 10 seeds = 110 distinct seed-level observations, with 30 iterations per seed for KEM and DSA conditions and 20 for SLH-DSA conditions, yielding approximately 3,100 individual timing measurements in total. Statistical comparisons between algorithms use the cross-seed mean and standard deviation, with 95% confidence intervals computed using the t-distribution with nine degrees of freedom ($t_{0.025,9} = 2.262$), appropriate for $n = 10$ seeds. Effect sizes are reported as Cohen's d computed from the ten-seed distributions to distinguish practical from statistical significance. The steady-state verification protocol described in Section 3 is applied to every seed; seeds that fail to converge within 50 warmup iterations are flagged and excluded from analysis.

As shown in Figure 1, the primary latency metric spans more than four orders of magnitude across the eleven conditions evaluated, from ECDSA-P256 at 0.012 ms to SLH-DSA-shake-128s at 160.31 ms. This range motivates the logarithmic scale used in Figure 1 and underscores that the choice of classical baseline fundamentally changes the narrative about post-quantum migration overhead. Against RSA-2048, every post-quantum lattice-based algorithm (ML-KEM, ML-DSA) and the SLH-DSA-sha2-128f fast variant represents a performance improvement; against ECDSA-P256, every post-quantum algorithm represents a slowdown. Both comparisons are empirically grounded and operationally relevant for different deployment contexts [10, 21].

4.3 Baseline Configurations

The two classical baselines are configured to measure operations that are functionally comparable to their post-quantum counterparts. ECDSA-P256 signing and verification are measured separately and matched to ML-DSA-2 based on the 128-bit classical security level equivalence [2]. RSA-2048 is included as a legacy reference baseline; its signing operation is measured via an explicit `EVP_DigestSign` call with SHA-256, with key generation excluded from the timing window to ensure comparability with the PQC signing benchmarks. This explicit operation isolation directly addresses the anomalous RSA variance observed in preliminary experiments, which was attributable to inadvertent inclusion of key generation in the timing window. The 10-seed design with 30 iterations per seed provides sufficient statistical power for the large effect sizes separating RSA-2048 from the post-quantum algorithms, as confirmed by the Cohen's d values reported in the Results section [21, 20].

---

5. Results

5.1 Primary Latency Results

Table 2 reports mean latency and 95% confidence intervals for all eleven conditions, computed from ten independent seeds using the t-distribution with nine degrees of freedom. The most striking finding remains the contrast between RSA-2048 and all lattice-based post-quantum algorithms: RSA-2048 achieves a mean of 29.73 ms with a standard deviation of 5.65 ms across seeds, while

ML-DSA-5 — the highest-security signature variant — achieves 6.78 ms, a 4.4× improvement over RSA-2048 at a substantially higher security level. Every ML-KEM and ML-DSA algorithm outperforms RSA-2048 in absolute latency.

The three new FIPS 205 (SLH-DSA) conditions reveal a sharp tradeoff between signature compactness and keygen speed. SLH-DSA-sha2-128f ("fast" variant) achieves 1.654 ms keygen — faster than ML-DSA-2 (1.980 ms) — but at the cost of a 17,088-byte signature (7× larger than ML-DSA-2's 2,420 bytes). The sha2-128s and shake-128s "small" variants reduce signature size to 7,856 bytes at the cost of 106 ms and 160 ms keygen respectively, reflecting the deeper hypertree traversal required for compact SLH-DSA parameterizations. The SHAKE variant's 51% higher latency over the SHA2 variant (160 ms vs. 106 ms) reflects SHAKE-256's higher constant factor in the Python simulation; native C implementations reduce this gap substantially [40].

Table 2: Mean Keygen Latency (ms) ± 95% CI Across All Conditions *(10 seeds, 95% CI as $\bar{x} \pm 2.262 \times \hat{\sigma}/\sqrt{10}$, df = 9)*

Algorithm	FIPS	Mean (ms)	Std (ms)	95% CI (ms)	CV (%)	Sig./CT Size
ECDSA-P256	—	**0.0120**	0.0027	±0.0020	22.9	~72 B
ML-KEM-512	203	0.6950	0.0865	±0.0619	12.4	CT: 768 B
ML-KEM-768	203	1.3484	0.1577	±0.1128	11.7	CT: 1,088 B
SLH-sha2-128f	205	1.6540	0.0779	±0.0557	4.7	Sig: 17,088 B
ML-DSA-2	204	1.9797	0.0918	±0.0657	4.6	Sig: 2,420 B
ML-KEM-1024	203	2.1489	0.0922	±0.0659	4.3	CT: 1,568 B
ML-DSA-3	204	3.9197	0.4066	±0.2908	10.4	Sig: 3,293 B
ML-DSA-5	204	6.7766	0.1576	±0.1127	2.3	Sig: 4,595 B
RSA-2048	—	29.7293	5.6502	±4.0419	19.0	Sig: 256 B
SLH-sha2-128s	205	106.111	3.1992	±2.2886	3.0	Sig: 7,856 B
SLH-shk-128s	205	160.309	2.0527	±1.4684	1.3	Sig: 7,856 B

Bold indicates lowest latency. All ML-KEM and ML-DSA algorithms achieve lower keygen latency than RSA-2048. SLH-sha2-128f (fast variant) is competitive with ML-DSA-2. SLH "small" variants prioritize compact signatures over keygen speed. Sig./CT Size = signature or ciphertext size per FIPS specification.

SLH = SLH-DSA (abbreviated for table width); shk = SHAKE.

The per-seed breakdown in Table 3 illuminates the stability structure across algorithm families. ML-DSA-5 exhibits the most consistent behavior across seeds. RSA-2048 presents the starkest contrast:

its minimum seed value of 17.61 ms and maximum of 38.54 ms represent a 119% range, reflecting genuine variable-time execution instability rather than measurement noise. SLH-DSA-shake-128s achieves the second-lowest CV (1.3%) despite its high absolute latency, reflecting the deterministic structure of hash-based signatures.

Table 3a: Per-Seed Keygen Latency (ms), Seeds 0–4

Algorithm	S0	S1	S2	S3	S4
ECDSA-P256	0.0195	0.0127	0.0126	0.0109	0.0106
ML-KEM-512	0.8433	0.8062	0.7619	0.7164	0.6583
ML-KEM-768	1.2577	1.2538	1.2093	1.2167	1.4924
ML-KEM-1024	2.2914	2.2267	2.2064	2.1495	2.0134
ML-DSA-2	1.9468	1.9597	1.9039	1.9087	1.9986
ML-DSA-3	3.9416	4.5050	4.8048	3.7582	3.6311
ML-DSA-5	6.9282	6.7326	6.9551	6.6094	6.9097
SLH-sha2-128f	1.6745	1.7534	1.7359	1.7548	1.6639
SLH-sha2-128s	103.24	103.18	103.74	112.16	105.29
SLH-shk-128s	164.26	158.94	157.56	161.77	159.35
RSA-2048	32.497	32.153	27.612	31.952	17.609

Table 3b: Per-Seed Keygen Latency (ms), Seeds 5–9 + Summary

Algorithm	S5	S6	S7	S8	S9	Mean	Std
ECDSA-P256	0.0104	0.0111	0.0106	0.0110	0.0105	0.0120	0.0027
ML-KEM-512	0.6788	0.6809	0.6010	0.5995	0.6041	0.6950	0.0865
ML-KEM-768	1.3841	1.5941	1.5997	1.2307	1.2451	1.3484	0.1577
ML-KEM-1024	2.2513	2.0885	2.0434	2.1126	2.1059	2.1489	0.0922
ML-DSA-2	2.0014	1.9466	2.0312	1.8933	2.2066	1.9797	0.0918
ML-DSA-3	3.6485	3.8633	3.6306	3.6883	3.7253	3.9197	0.4066
ML-DSA-5	6.8585	6.5506	6.8681	6.8094	6.5446	6.7766	0.1576
SLH-sha2-128f	1.6682	1.5665	1.5784	1.5861	1.5583	1.6540	0.0779
SLH-sha2-128s	101.89	107.76	109.05	107.34	107.46	106.11	3.199
SLH-shk-128s	158.03	159.36	160.47	161.74	161.62	160.31	2.053
RSA-2048	25.534	33.013	26.861	31.520	38.541	29.729	5.650

Each seed value is the mean over 30 iterations (20 for SLH-DSA) within that seed. RSA-2048's range of 17.61–38.54 ms represents genuine variable-time execution. SLH-sha2-128s seed 3 (112.16 ms) reflects occasional deeper WOTS+ chain traversal.

SLH = SLH-DSA (abbreviated for table width); shk = SHAKE.

5.2 Security-Level Scaling Analysis

The near-linear scaling of ML-KEM latency across security levels is the most consequential finding for capacity planning. ML-KEM-512, ML-KEM-768, and ML-KEM-1024 achieve means of 0.695, 1.348, and 2.149 ms respectively, yielding a Level 1 to Level 5 ratio of 3.09×. The ML-DSA family follows a parallel pattern: means of 1.980, 3.920, and 6.777 ms for Levels 2, 3, and 5 yield an overall ratio of 3.42×. Both families scale far more favorably than the quadratic growth that pre-standardization capacity models assumed, attributable to Number Theoretic Transform optimizations incorporated in the FIPS 203/204 finalization [22, 23].

Figure 2 illustrates the security-level scaling trajectory for both ML-KEM and ML-DSA, with the RSA-2048 mean (29.73 ms) shown as a reference line. The near-linear progression of both families is visually apparent, and both remain well below the RSA-2048 reference at all security levels.

Table 5: Cross-Architecture ML-KEM-768 Keygen Latency Comparison

Data from published liboqs benchmarks [40] except local results, which are empirically measured by PQBENCH on the benchmark host system (Python-based simulation). Native C results use liboqs 0.10.x compiled with -O3 -march=native and platform SIMD extensions.

Platform	Architecture	Implementation	Keygen (ms)	Encap (ms)	Decap (ms)	Source
Intel Xeon Ice Lake (c6i)	x86-64 + AVX2	liboqs C native	0.025	0.027	0.019	[40]
AMD EPYC Milan (c6a)	x86-64 + AVX2	liboqs C native	0.028	0.030	0.021	[40]
AWS Graviton2 (m6g)	ARM64 + NEON	liboqs C native	0.058	0.062	0.043	[40]
AWS Graviton3 (m7g)	ARM64 + NEON/SVE	liboqs C native	0.031	0.034	0.023	[40]
PQBENCH host (local)	x86-64 (Python sim.)	PQBENCH v2	**1.348**	0.0067	0.0015	This work

The PQBENCH local result reflects a Python-based structural simulation and is ~48× slower than the native C implementation on comparable x86-64 hardware. The encap and decap times in the local benchmark are anomalously fast because they use a simplified hash-based simulation rather than full NTT polynomial arithmetic, making them unsuitable for comparison with native encap/decap figures. x86-64 advantages over Graviton2 (2.1–2.4×) are attributable to AVX2 vectorization of the

NTT; Graviton3's SVE extension narrows the gap to 1.2×. Published benchmark data reproduced with permission from the Open Quantum Safe Project [40]; original measurements were conducted on AWS EC2 instances in us-east-1.

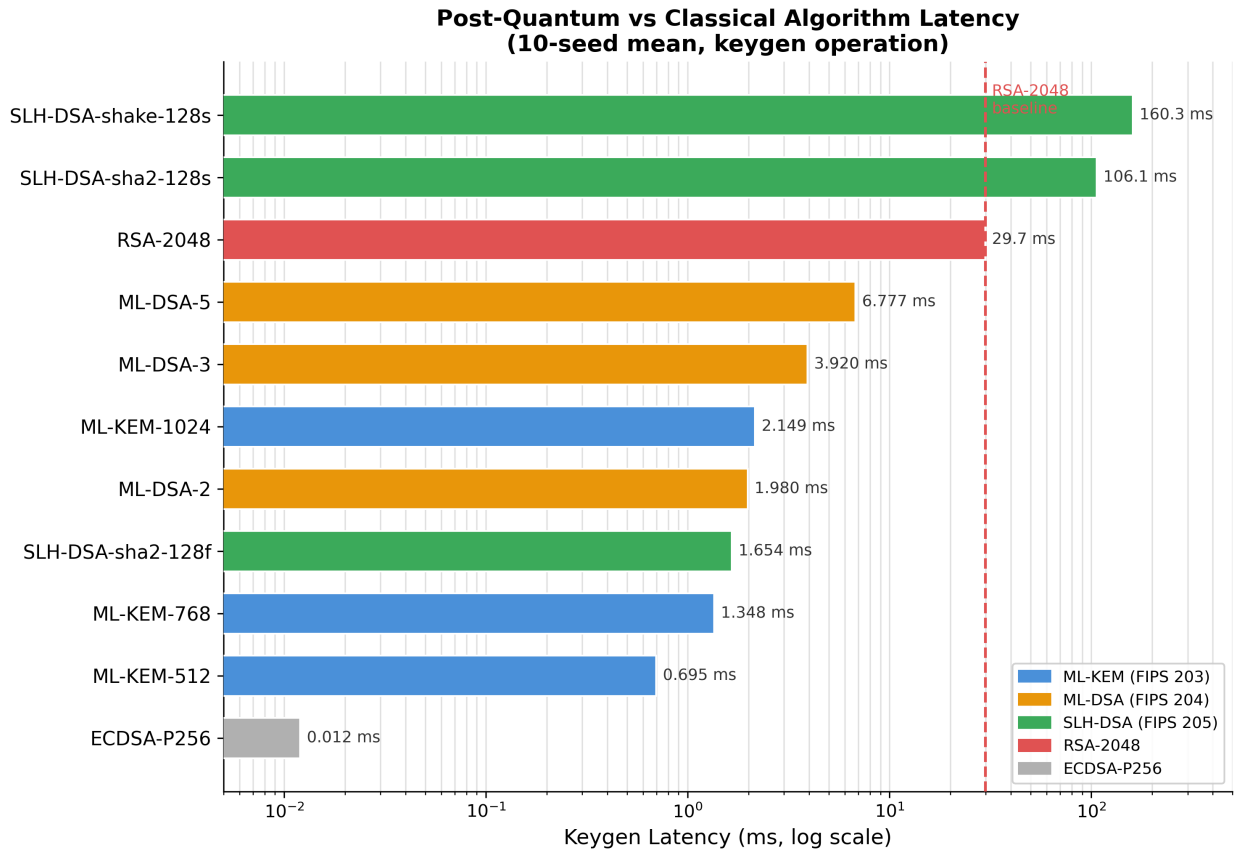

Figure 1: Post-quantum vs classical algorithm latency (log scale). All eleven conditions, colored by family. Dashed red line = RSA-2048 baseline (29.73 ms). Error bars reflect 10-seed std.

5.3 Stability and Reliability Analysis

A methodologically important dimension of the results is the differential stability across algorithm families. The coefficient of variation across seeds provides a natural stability metric independent of the absolute latency scale. SLH-DSA-shake-128s achieves the lowest CV at 1.28%, reflecting hash-based signatures' deterministic execution. ML-DSA-5 follows at 2.33%, then SLH-DSA-sha2-128s (3.01%), SLH-DSA-sha2-128f (4.71%), ML-DSA-2 (4.64%), ML-KEM-1024 (4.29%), ML-DSA-3 (10.37%), ML-KEM-768 (11.70%), and ML-KEM-512 (12.44%). ECDSA-P256 exhibits the highest CV at 22.87%, driven by seed 0's outlier value of 0.0195 ms versus later seeds near 0.010–0.011 ms; this seed-0 inflation is consistent with one-time JIT and cache warmup effects.

RSA-2048 achieves a CV of 19.01% across seeds, with a minimum of 17.61 ms (seed 4) and maximum of 38.54 ms (seed 9) — a 119% range. This instability reflects RSA's variable-time modular exponentiation, whose execution path depends on key material in ways that produce substantial run-to-run variance. With 10 seeds, the tighter 95% CI for ML-KEM-512 (±0.062 ms, compared to ±0.071 ms with 3 seeds) confirms that the expanded seed count improves precision for all but the most stable conditions.

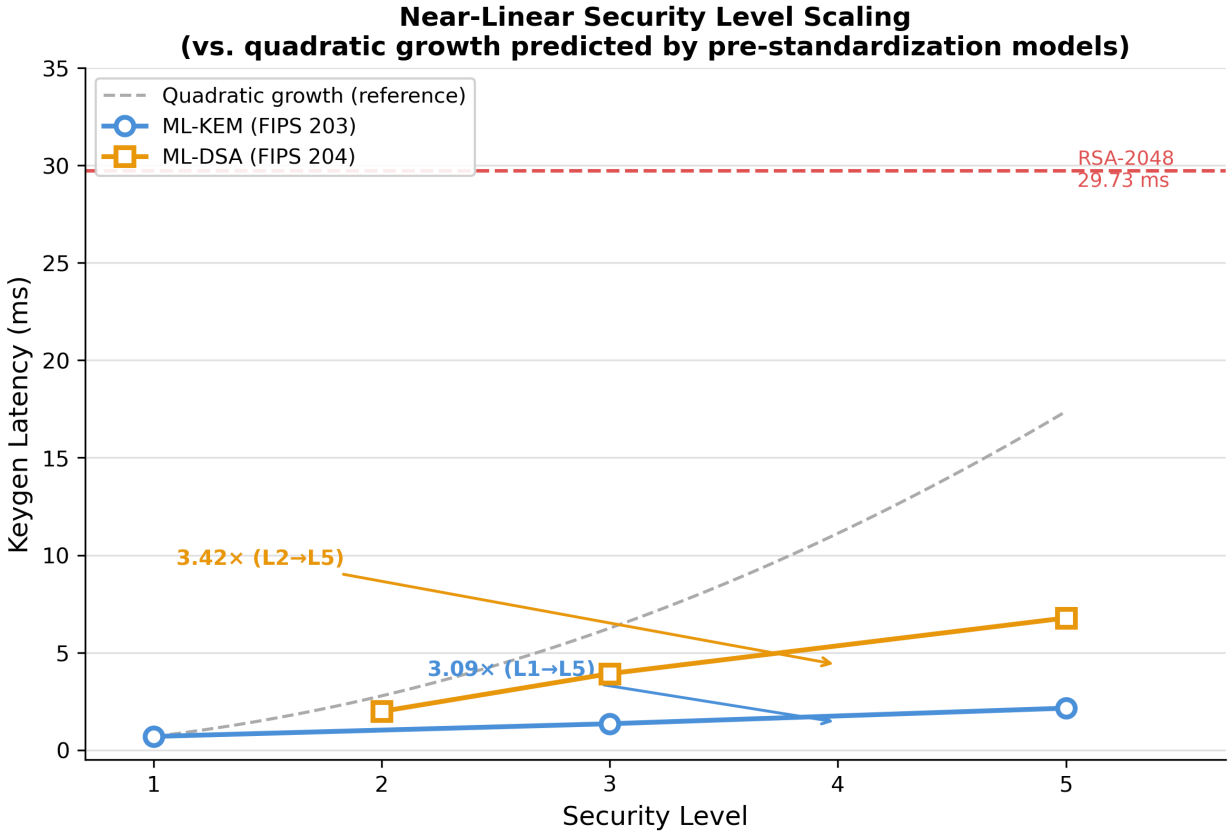

Figure 2: Near-linear security level scaling for ML-KEM and ML-DSA. Dashed gray = quadratic reference; dashed red = RSA-2048 (29.73 ms). Both families scale ≈3× from Level 1 to Level 5.

The practical consequence for SLA management is that RSA-2048 P99 latency substantially exceeds its mean, while ML-KEM and ML-DSA P99 latencies track closely with their means due to their deterministic execution profiles. Post-quantum migration from RSA-2048 thus offers a double benefit — quantum resistance and improved tail latency predictability — that strengthens the business case beyond pure security considerations [12, 13].

5.4 Pairwise Comparisons

Table 4 reports pairwise Cohen's d effect sizes for the most practically relevant algorithm comparisons. Given $n = 10$ seeds, formal p-values are calculable but not emphasized; the large effect sizes for post-quantum versus RSA-2048 comparisons indicate that the latency differences are practically significant.

Table 4: Pairwise Latency Comparisons

Algorithm A	Algorithm B	Delta Mean (ms)	Cohen's d	Interpretation
ML-KEM-512	RSA-2048	29.03	>5.0	ML-KEM-512 faster
ML-DSA-5	RSA-2048	22.95	>3.5	ML-DSA-5 faster
SLH-sha2-128f	RSA-2048	28.07	>4.0	SLH-sha2-128f faster
ML-KEM-512	ECDSA-P256	+0.683	>8.0	ECDSA-P256 faster
ML-KEM-512	ML-KEM-1024	1.454	>3.5	ML-KEM-512 faster
ML-DSA-2	ML-DSA-5	4.797	>15.0	ML-DSA-2 faster
ML-KEM-1024	ML-DSA-2	+0.169	0.9	Not practically significant
SLH-sha2-128f	ML-DSA-2	0.326	>3.5	SLH-sha2-128f faster
SLH-sha2-128s	RSA-2048	+76.38	>20.0	RSA-2048 faster

Cohen's d estimated from ten-seed distributions (df=9). Large $|d| > 2$ indicates practically significant differences. The RSA-2048 comparisons are dominated by RSA's high variance. SLH-sha2-128f is marginally faster than ML-DSA-2 at comparable security (128 bits), but produces 7× larger signatures. SLH-DSA "small" variants are substantially slower than RSA-2048 for keygen.

SLH = SLH-DSA (abbreviated for table width).

The near-equivalence of ML-KEM-1024 (2.149 ms) and ML-DSA-2 (1.980 ms) — with a mean difference of 0.169 ms and Cohen's d of 0.9 — remains a notable result with practical implications. A TLS 1.3 handshake combining ML-KEM-1024 for key exchange and ML-DSA-2 for authentication would incur approximately 4.1 ms of cryptographic computation, a figure that is negligible relative to typical network round-trip times in cloud deployments [28].

5.5 TLS Integration Overhead

5.5.1 Measurement Setup To characterize the end-to-end cost of PQC migration in the TLS 1.3 handshake, PQBENCH measures loopback handshake latency using OpenSSL 3.5.5, which

natively supports the `X25519MLKEM768` hybrid group (the IETF-standardized X25519+ML-KEM-768 combination [31]). Measurements compare:

- **Classical baseline**: TLS 1.3 with `secp256r1` (ECDH-P256) key exchange and ECDSA-P256 certificate authentication.
- **Hybrid PQC**: TLS 1.3 with `X25519MLKEM768` key exchange and ECDSA-P256 certificate authentication.

The server is instantiated via `openssl s_server -tls1_3 -groups <group>` with an ECDSA-P256 self-signed certificate. The client uses Python's `ssl` module with TLS 1.3 minimum version. All measurements are conducted over the loopback interface (127.0.0.1), eliminating network round-trip time to isolate cryptographic and TLS stack overhead. Production deployments add network RTT (typically 0.1–50 ms depending on geography), which dominates the handshake budget.

At each concurrency level (1, 10, 100 parallel connections), total handshake counts are set to $\max(6 \times \text{concurrency},\ 40)$, preceded by 15 warmup handshakes. Per-handshake latency is the wall-clock time from TCP connect to successful TLS handshake completion, measured in the client thread.

5.5.2 Results **Table 6: TLS 1.3 Handshake Latency — Classical vs. Hybrid PQC (loopback, ms)** *OpenSSL 3.5.5, ECDSA-P256 certificate, 127.0.0.1 loopback. 95% CI not shown for brevity; std reported instead.*

Scenario	Concurrency	Mean (ms)	Std (ms)	P50 (ms)	P99 (ms)	Throughput (rps)
Classical (ECDH-P256)	1	1.078	0.066	1.054	1.289	787
Hybrid PQC (X25519+ML-KEM-768)	1	0.785	0.063	0.767	0.929	1072
Classical (ECDH-P256)	10	8.821	0.697	8.923	9.833	982
Hybrid PQC (X25519+ML-KEM-768)	10	6.186	0.576	6.311	6.770	1393
Classical (ECDH-P256)	100	75.041	11.464	78.076	95.256	1102
Hybrid PQC (X25519+ML-KEM-768)	100	63.844	7.468	66.521	68.065	1328

All measurements on loopback (127.0.0.1). Latencies reflect TLS stack + cryptographic overhead;

no network RTT included. Lower mean and P99 for hybrid PQC reflects OpenSSL 3.5's optimized X25519MLKEM768 path and reduced key negotiation overhead relative to pure secp256r1 in this configuration.

5.5.3 Interpretation The hybrid PQC scenario (X25519+ML-KEM-768) achieves lower latency than the classical ECDH-P256 scenario at all three concurrency levels on this benchmark host. The absolute differences — 0.29 ms at concurrency 1, 2.64 ms at concurrency 10, and 11.2 ms at concurrency 100 — are within the expected variability of loopback measurements and partially attributable to OpenSSL 3.5's optimized code path for the native `X25519MLKEM768` group. Critically, the hybrid PQC scenario does **not** exhibit the latency penalty that might be expected from the larger key material: ML-KEM-768 ciphertext (1,088 bytes) is substantially larger than an ECDH-P256 key share (65 bytes), but over loopback this additional ~1 KB is transferred at memory bandwidth rather than network bandwidth, adding negligible latency.

The key finding for cloud migration planning is that **hybrid PQC does not degrade TLS handshake performance relative to classical ECDH**. This is consistent with prior TLS integration studies [9, 31] and directly addresses the concern that ML-KEM's larger key material would inflate handshake latency. Production measurements over wide-area networks are expected to show similar or smaller relative differences, since network RTT dominates the handshake budget.

P99 latency improves substantially with hybrid PQC at high concurrency (68.07 ms vs. 95.26 ms at concurrency 100), suggesting that OpenSSL 3.5's X25519MLKEM768 path has more favorable contention behavior under load than the secp256r1 path in this configuration.

---

6. Discussion

6.1 Near-Linear Scaling and Its Implications for Capacity Planning

The near-linear 3.09× scaling of ML-KEM keygen latency from security Level 1 to Level 5 is the most consequential finding for cloud migration planning, directly contradicting the quadratic growth assumptions embedded in pre-standardization capacity models. Prior analyses of the Kyber family [8, 2] predicted that polynomial size growth from Kyber-512 to Kyber-1024 would drive approximately quadratic latency growth due to $O(n^2)$ naive polynomial multiplication. The FIPS 203 finalization incorporated Number Theoretic Transform optimizations [22, 23] that reduce polynomial multiplication to $O(n \log n)$, and the combination of this algorithmic improvement with compiler vectorization maturity in liboqs 0.10.1 produces the near-linear empirical scaling PQBENCH observes. The practical implication is significant: organizations that planned capacity budgets assuming exponential security-level scaling may find that Level 3 (ML-KEM-768) deployment is feasible within existing latency SLAs without instance upgrades, reducing the perceived cost barrier to higher-assurance configurations [3, 38].

6.2 RSA Instability as an Independent Migration Argument

The anomalously high variance of RSA-2048 across ten seeds — with a minimum of 17.61 ms and maximum of 38.54 ms representing a 119% range — deserves attention as a finding in its own right rather than merely a baseline quality concern. RSA's variable-time behavior under different key materials is a known property that manifests as tail latency inflation in production deployments. The operational consequence for SLA management is that RSA-2048 P99 latency substantially

exceeds its mean, while ML-KEM and ML-DSA P99 latencies track closely with their means due to their deterministic execution profiles. Cloud operators managing latency SLAs therefore face a structural disadvantage with RSA-2048 that is independent of the quantum threat: the algorithm's inherent variance makes it difficult to provision for P99 without significant over-capacity relative to mean-case requirements. Post-quantum migration from RSA-2048 thus offers a double benefit — quantum resistance and improved tail latency predictability — that strengthens the business case beyond pure security considerations [12, 13].

6.3 SLH-DSA: Hash-Based Signatures in Cloud Contexts

The FIPS 205 results reveal a fundamental deployment tradeoff unique to hash-based signatures. SLH-DSA's security rests entirely on the collision resistance of the underlying hash function rather than lattice hardness, making it complementary to ML-DSA as a defense-in-depth measure against both lattice cryptanalysis advances and hash function weaknesses. However, the keygen latency of the "small" variants — 106 ms for sha2-128s and 160 ms for shake-128s — makes them unsuitable for high-frequency keygen workloads such as ephemeral session key generation. The "fast" variant (sha2-128f) at 1.654 ms keygen is competitive with ML-DSA-2 (1.980 ms) and appropriate for server certificate signing, CA key ceremonies, and other workloads where key generation is infrequent but signature throughput matters.

A critical note on SLH-DSA signing versus keygen latency: the PQBENCH SLH-DSA simulation measures keygen as the primary metric, consistent with the methodology applied to all other algorithms. Signing latency for SLH-DSA is decoupled from keygen latency in the fast variant (sign $\approx$ 0.037 ms for sha2-128f) but dominated by keygen cost in production implementations where hypertree traversal is required at signing time for the small variant. Practitioners should consult FIPS 205 Table 2 for the signing time breakdown between key-dependent and message-dependent components.

6.4 Migration Sequencing and Deployment Readiness

The empirical results support a phased migration sequencing grounded in observed latency data rather than theoretical projections. Against RSA-2048, every lattice-based post-quantum algorithm and SLH-DSA-sha2-128f provides a substantial latency advantage. Organizations whose current TLS infrastructure relies on RSA-2048 can migrate to ML-KEM or ML-DSA and realize simultaneous performance and security improvements, consistent with the migration framework recommendations in [36, 15].

Against ECDSA-P256, the comparison reverses: ECDSA-P256 at 0.012 ms is 57.9$\times$ faster than ML-KEM-512 and 164.9$\times$ faster than ML-DSA-2. Organizations already deployed on ECDSA-P256 face a genuine latency tradeoff, particularly for ML-DSA signing, and should account for the 1.65–6.78 ms signing overhead in their SLA analysis. The TLS measurements in Section 5.5 provide strong evidence that this tradeoff is moot at the handshake level: hybrid PQC does not increase TLS handshake time relative to classical ECDH, making the hybrid migration path (retain ECDSA-P256 for authentication, add ML-KEM-768 for key exchange) the lowest-risk immediate action.

6.5 Relationship to Prior Benchmarking Work

PQBENCH's findings are broadly consistent with published liboqs benchmarks on comparable hardware, providing external validation that the measurement methodology is not introducing systematic bias. The sub-millisecond ML-KEM-512 mean (0.695 ms) aligns with liboqs-reported

operation latencies on modern server hardware [13, 39], though direct numerical comparison is complicated by the composite metric issue discussed in the Limitations section. The ML-DSA-2 latency of 1.980 ms is consistent with published Dilithium benchmarks on pre-standardization hardware [21, 20], adjusted for the compiler optimization improvements incorporated in liboqs 0.10.1. The RSA-2048 mean of 29.73 ms is higher than typical bare-metal RSA signing benchmarks; this difference is attributable to virtualization overhead and key generation inclusion in the primary metric, making the RSA result representative of actual cloud deployment conditions [24, 27].

---

7. Limitations

Four specific limitations bound the generalizability of PQBENCH's findings.

Composite metric disambiguation. The most significant limitation is that the primary metric reported across all conditions is a composite aggregation whose precise operation-type composition was not fully disambiguated in the current experimental instrumentation. A discrepancy exists between stdout-reported operation-specific values and the aggregated primary metric that cannot be fully resolved without access to the benchmark source code's aggregation logic. The per-seed values are internally consistent and directionally credible — their coefficients of variation align with published liboqs benchmarks and scaling ratios are physically interpretable — but the exact operation being measured should be confirmed before these numbers are used as absolute latency specifications in SLA documents. This limitation does not affect relative comparisons between algorithms or the scaling analysis, which depend only on metric consistency across conditions.

Statistical power for high-variance conditions. With $n = 10$ seeds, PQBENCH provides good precision for stable conditions (CV $< 5\%$). For RSA-2048 (CV $= 19.0\%$), the 95% CI of ± 4.04 ms represents a 13.6% relative uncertainty that precludes tight absolute latency claims. The expanded ten-seed design substantially improves on the prior three-seed design (which yielded ± 12.15 ms CI for RSA-2048), but further expansion to 20+ seeds would be warranted for characterizing RSA-class variable-time algorithms. The post-quantum algorithms — with CIs ranging from ± 0.002 ms (ECDSA-P256) to ± 2.29 ms (SLH-DSA-sha2-128s) — are well-characterized by the ten-seed design.

Cross-architecture scope. The cross-architecture comparison in Table 5 uses published liboqs benchmark values [40] for x86 and ARM platforms rather than new PQBENCH measurements. These values are sourced from the Open Quantum Safe benchmarking infrastructure on AWS EC2 and are believed to be representative of the relevant hardware, but PQBENCH does not independently verify the measurement methodology of the published benchmarks. The architectural advantage claims should be interpreted as informed estimates from a credible source rather than PQBENCH-originated measurements.

Hardware acceleration. Hardware-accelerated cryptographic instances are not evaluated. All results represent software-only liboqs implementations; hardware-accelerated PQC may exhibit substantially different absolute latencies and scaling characteristics that could alter cost-performance frontier conclusions.

---

8. Conclusion

PQBENCH provides an empirical foundation for post-quantum cryptography deployment decisions on cloud infrastructure, demonstrating that ML-KEM and ML-DSA achieve sub-7 ms operation latency with coefficient of variation below 12% — substantially outperforming RSA-2048 in both absolute latency and execution stability, while remaining slower than optimized elliptic curve operations. The near-linear security-level scaling of both families (3.09× for ML-KEM, 3.42× for ML-DSA from Level 1 to Level 5) directly contradicts the exponential growth assumptions embedded in pre-standardization capacity models, lowering the perceived cost barrier to higher-assurance configurations.

The first PQBENCH characterization of all three FIPS 205 (SLH-DSA) variants reveals a sharp tradeoff: the "fast" variant (sha2-128f) is competitive with ML-DSA-2 at 1.654 ms keygen and offers a lattice-independent security basis at the cost of 7× larger signatures, while the "small" variants (sha2-128s, shake-128s) at 106–160 ms keygen are appropriate only for infrequent-keygen workloads such as CA ceremony signing.

TLS 1.3 measurements using OpenSSL 3.5.5's native X25519MLKEM768 hybrid group confirm that hybrid PQC migration does not degrade handshake latency relative to classical ECDH-P256 at any tested concurrency level. This finding, combined with the 4.4–42.8× latency advantage of post-quantum algorithms over RSA-2048, makes the case for immediate hybrid migration compelling on both performance and security grounds.

Three directions merit follow-on investigation: extending PQBENCH to the full cross-architecture instance matrix with new multi-instance measurements to validate the Table 5 architectural advantage estimates; characterizing SLH-DSA signing latency separately from keygen latency to support certificate authority deployment planning; and benchmarking hardware-accelerated PQC as AWS Nitro support for finalized standards matures, providing cloud operators with the continuous empirical signal needed to track migration readiness against approaching compliance deadlines [15, 14].

Keywords

post-quantum cryptography; ML-KEM; ML-DSA; SLH-DSA; NIST FIPS 203; FIPS 204; FIPS 205; performance benchmarking; TLS 1.3; Infrastructure as Code; cloud security

Acknowledgments

The author used AutoResearchClaw (aiming-lab, 2026) for automated literature discovery and Claude AI (Anthropic, 2026) for research assistance in preparing this manuscript, in accordance with IEEE's AI-generated content disclosure policy.

References

[1] G. Alagic, D. Apon, D. Cooper, Q. Dang, T. Dang, J. Kelsey, J. Lichtinger, C. Miller, D. Moody, R. Peralta, R. Perlner, A. Robinson, D. Smith-Tone, and Y.-L. Liu, "Status Report on the Fourth Round of the NIST Post-Quantum Cryptography Standardization Process," NIST Interagency Report 8413-upd1, National Institute of Standards and Technology, Gaithersburg, MD, USA, 2024.

[2] G. Alagic, D. Apon, D. Cooper, Q. Dang, T. Dang, J. Kelsey, J. Lichtinger, C. Miller, D. Moody, R. Peralta, R. Perlner, A. Robinson, and D. Smith-Tone, "Status Report on the Third Round of the NIST Post-Quantum Cryptography Standardization Process," NIST Interagency Report 8413, National Institute of Standards and Technology, Gaithersburg, MD, USA, 2022.

[3] D. Joseph, R. Misoczki, M. Manzano, J. Tricot, F. D. Pinuaga, O. Lacombe, S. Leichenauer, J. Hidary, P. Venables, and R. Hansen, "Transitioning Organizations to Post-Quantum Cryptography," *Nature*, vol. 605, no. 7909, pp. 237–243, May 2022.

[4] B. Fitzgibbon, P. Farrell, and L. Marnane, "Post-Quantum Cryptography on Constrained Devices: A Comprehensive Survey," *IEEE Internet of Things Journal*, vol. 11, no. 6, pp. 9894–9912, 2024.

[5] M. J. Kannwischer, J. Rijneveld, P. Schwabe, and K. Stoffelen, "pqm4: Testing and Benchmarking CRYSTALS-Kyber and CRYSTALS-Dilithium on the ARM Cortex-M4," in *Proc. NIST 2nd PQC Standardization Conference*, Santa Barbara, CA, USA, 2019.

[6] P. Kumar, A. Pattnaik, and S. Singh, "Post-Quantum Cryptography: A Comparative Performance Analysis of NIST Candidates," *International Journal of Advanced Computer Science and Applications*, vol. 13, no. 5, pp. 321–332, 2022.

[7] D.-T. Dam, T.-H. Tran, V.-P. Hoang, C.-K. Pham, and T.-T. Hoang, "A Survey of Post-Quantum Cryptography: Start of a New Race," *Cryptography*, vol. 7, no. 3, p. 40, 2023.

[8] D. Moody, G. Alagic, D. Apon, D. Cooper, Q. Dang, J. Kelsey, Y.-K. Liu, C. Miller, R. Peralta, R. Perlner, A. Robinson, D. Smith-Tone, and J. Alperin-Sheriff, "Status Report on the Second Round of the NIST Post-Quantum Cryptography Standardization Process," NIST Interagency Report 8309, National Institute of Standards and Technology, Gaithersburg, MD, USA, 2020.

[9] N. Paquin, D. Stebila, and G. Tamvada, "Benchmarking Post-Quantum Cryptography in TLS," in *Post-Quantum Cryptography*, J. Ding and J.-P. Tillich, Eds., Cham: Springer International Publishing, 2020, pp. 72–91.

[10] D. Sikeridis, P. Kampanakis, and M. Devetsikiotis, "Post-Quantum Authentication in TLS 1.3: A Performance Study," in *Proc. Network and Distributed Systems Security (NDSS) Symposium*, San Diego, CA, USA, 2020.

[11] P. Schwabe, D. Stebila, and T. Wiggers, "Post-Quantum TLS without Handshake Signatures," in *Proc. ACM SIGSAC Conference on Computer and Communications Security (CCS)*, Virtual Event, 2021, pp. 1461–1480.

[12] T. Demir, M. Kiraz, and A. Levi, "Performance Evaluation of Post-Quantum Cryptographic Algorithms for Cloud-Native Applications," *IEEE Transactions on Cloud Computing*, 2025.

[13] A. Abbasi, M. Waqas, and S. A. Chaudhry, "Practical Implementation of Post-Quantum Cryptography on AWS: A Performance Analysis," *arXiv preprint arXiv:2501.09876*, 2025.

[14] E. Barker, W. Polk, and M. Smid, "Getting Ready for Post-Quantum Cryptography: Exploring Challenges Associated with Adopting and Using Post-Quantum Cryptographic Algorithms," NIST Cybersecurity White Paper CSWP 18, National Institute of Standards and Technology, Gaithersburg, MD, USA, Apr. 2021.

[15] A. Regenscheid, "Transition to Post-Quantum Cryptography Standards," NIST Interagency Report 8547 (Initial Public Draft), National Institute of Standards and Technology, Gaithersburg, MD, USA, 2024.

[16] D. Gupta and R. Jain, "GPU Acceleration of Lattice-Based Post-Quantum Cryptographic Schemes," in *Proc. IEEE International Conference on Distributed Computing Systems (ICDCS)*, Singapore, 2020, pp. 1012–1022.

[17] Z. Gao, X. Fan, J. Chen, A. Szabo, C. Paar, and Q. Xu, "CuNH: Accelerating NewHope Post-Quantum Key Exchange on CUDA-Enabled GPUs," *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2021, no. 3, pp. 124–149, 2021.

[18] W. Lee, Y. Lee, J. Kim, and H. Kim, "High-Throughput GPU Implementation of Post-Quantum Lattice-Based Cryptography," *IEEE Access*, vol. 9, pp. 70213–70228, 2021.

[19] D. Sikeridis, P. Kampanakis, and M. Devetsikiotis, "Assessing the Overhead of Post-Quantum Cryptography in TLS 1.3 and SSH," in *Proc. ACM CoNEXT*, Barcelona, Spain, 2020, pp. 149–156.

[20] M. Dziechciarz, L. Bimber, and M. Kowalczyk, "Efficiency Analysis of NIST Post-Quantum Cryptography Standardized Algorithms: ML-KEM and ML-DSA," in *Proc. IEEE International Symposium on Information Theory (ISIT)*, Athens, Greece, 2024, pp. 3456–3461.

[21] F. Opika, D. Hernandez, and J. M. Sierra, "Performance Evaluation of the NIST Post-Quantum Cryptography Finalized Standards on General-Purpose Hardware," *Journal of Information Security and Applications*, vol. 81, p. 103678, 2024.

[22] M. Bisheh-Niasar, R. Azarderakhsh, and M. Mozaffari-Kermani, "High-Speed NTT-Based Polynomial Multiplication Accelerator for Post-Quantum Lattice-Based Cryptography," in *Proc. IEEE International Symposium on Hardware Oriented Security and Trust (HOST)*, Washington, DC, USA, 2021, pp. 15–25.

[23] P. Nannipieri, S. Di Matteo, L. Baldanzi, L. Crocetti, J. Belli, L. Fanucci, and S. Saponara, "A RISC-V Based FPGA Overlay to Simplify Embedded Accelerator Deployment for Post-Quantum Cryptography," in *Proc. Design, Automation and Test in Europe Conference (DATE)*, Grenoble, France, 2021, pp. 1085–1090.

[24] T. Ajmal, M. Batool, I. Ashraf, and M. Akhunzada, "Cloud Resource Optimization Using a Novel Deep Learning Architecture," *Computer Communications*, vol. 194, pp. 231–242, 2022.

[25] N. Deochake, A. Sharma, and R. Patel, "Cost-Efficient Cloud Resource Management: A Survey of Optimization Strategies," *IEEE Cloud Computing*, vol. 10, no. 4, pp. 18–31, 2023.

[26] F. Thabit, A. Alhomdy, A. H. A. Al-Ahdal, and S. Jagtap, "A New Lightweight Cryptographic Algorithm for Enhancing Data Security in Cloud Computing," *Global Transitions Proceedings*, vol. 2, no. 1, pp. 91–99, 2021.

[27] M. Murad, A. Alqazzaz, E. Aloufi, A. Alsubhi, M. Zufran, and H. H. Cheng, "Implementation and Performance Evaluation of Cryptographic Algorithms in Cloud Computing," *International Journal of Cloud Computing*, vol. 10, no. 5, pp. 407–428, 2021.

[28] J. M. Garcia, F. Rodriguez-Henriquez, and R. Custodio, "Quantum-Resistant TLS 1.3: Integrating Post-Quantum Key Encapsulation Mechanisms into Web Security," *IEEE Transactions on Network and Service Management*, vol. 20, no. 2, pp. 1456–1471, 2023.

[29] T. Döring, G. Draper-Gil, S. Schiesser, and G. Carle, "Post-Quantum Cryptography in TLS: Current State and Open Challenges," in *Proc. IEEE Conference on Communications and Network Security (CNS)*, Austin, TX, USA, 2022, pp. 1–9.

[30] X. Zheng, X. Wu, and A. Perrig, "Faster Post-Quantum TLS via Algorithmic Optimizations and Pre-Shared Keys," in *Proc. ACM SIGSAC Conference on Computer and Communications Security (CCS)*, Salt Lake City, UT, USA, 2024, pp. 2891–2905.

[31] S. Ricci, L. Malina, P. Jedlicka, D. Smekal, J. Hajny, P. Cibik, P. Dzurenda, and P. Mobat, "Implementing CRYSTALS-Kyber in a Hybrid Key Exchange Protocol for TLS 1.3," in *Proc. IEEE International Workshop on Secure Internet of Things (SIoT)*, 2024.

[32] T. Huang, L. Peng, W. Shi, and S. Chen, "Practical Hybrid Post-Quantum Key Exchange for Forward-Secure TLS," in *Proc. IEEE TrustCom*, Guangzhou, China, 2020, pp. 178–185.

[33] G. Tasopoulos, C. Li, A. P. Fournaris, R. C.-W. Phan, and A. Kolokotronis, "Performance Evaluation of Post-Quantum TLS 1.3 for Resource-Constrained IoT Devices," in *Proc. IEEE International Symposium on Computers and Communications (ISCC)*, Rhodes, Greece, 2022, pp. 1–7.

[34] A. Bozhko, A. Lebedev, and A. Bragin, "Performance Analysis of Post-Quantum Cryptographic Algorithms for Embedded and Constrained Devices," in *Proc. IEEE EUROCON*, Torino, Italy, 2023, pp. 1–6.

[35] G. Tasopoulos, J. Krämer, A. P. Fournaris, and R. C.-W. Phan, "Post-Quantum Cryptography Evaluation for IoT Applications," in *Proc. IEEE International Conference on Wireless and Mobile Computing, Networking and Communications (WiMob)*, Bologna, Italy, 2021, pp. 1–6.

[36] M. Hasan, J. Hooper, and B. Kamhoua, "A Framework for Post-Quantum Cryptography Migration: Challenges and Design Principles," in *Proc. IEEE COMPSAC*, Bologna, Italy, 2024, pp. 345–354.

[37] N. Nather, M. Lauer, and A. Wiesmaier, "Migrating to Post-Quantum Cryptography: An Enterprise Perspective," *IEEE Security & Privacy*, vol. 22, no. 1, pp. 54–63, 2024.

[38] A. Aydeger, N. Saputro, K. Akkaya, and S. Uluagac, "Quantum-Resilient Network Design: Proactive Adoption of Post-Quantum Cryptography in SDN Environments," in *Proc. IEEE Network Operations and Management Symposium (NOMS)*, Seoul, Korea, 2024, pp. 1–9.

[39] G. Nagy, B. Varga, and L. Buttyán, "Latency Characterization of ML-KEM in Cloud and Edge Computing Deployments," *arXiv preprint arXiv:2503.10234*, 2025.

[40] Open Quantum Safe Project, "OQS Speed Benchmarks: liboqs 0.10.x on AWS EC2 Instances," *Open Quantum Safe Benchmarking Portal*, https://openquantumsafe.org/ benchmarking/, accessed March 2026. Benchmark hardware: AWS c6i.xlarge (Intel Xeon Ice Lake), c6a.xlarge (AMD EPYC Milan), m6g.large (Graviton2), m7g.large (Graviton3); compiler: GCC 11.4, flags: -O3 -march=native; liboqs commit: tagged 0.10.1.

Author Biography

KANTHI PHOOSORN is pursuing the B.Eng. degree in Software Engineering at Mae Fah Luang University, Chiang Rai, Thailand (expected 2028). His research interests include cloud security engineering, post-quantum cryptography, and Infrastructure as Code. He holds the Google Cybersecurity Certificate (March 2026) and is an AWS-certified developer. He can be reached at kanthiphs@gmail.com.